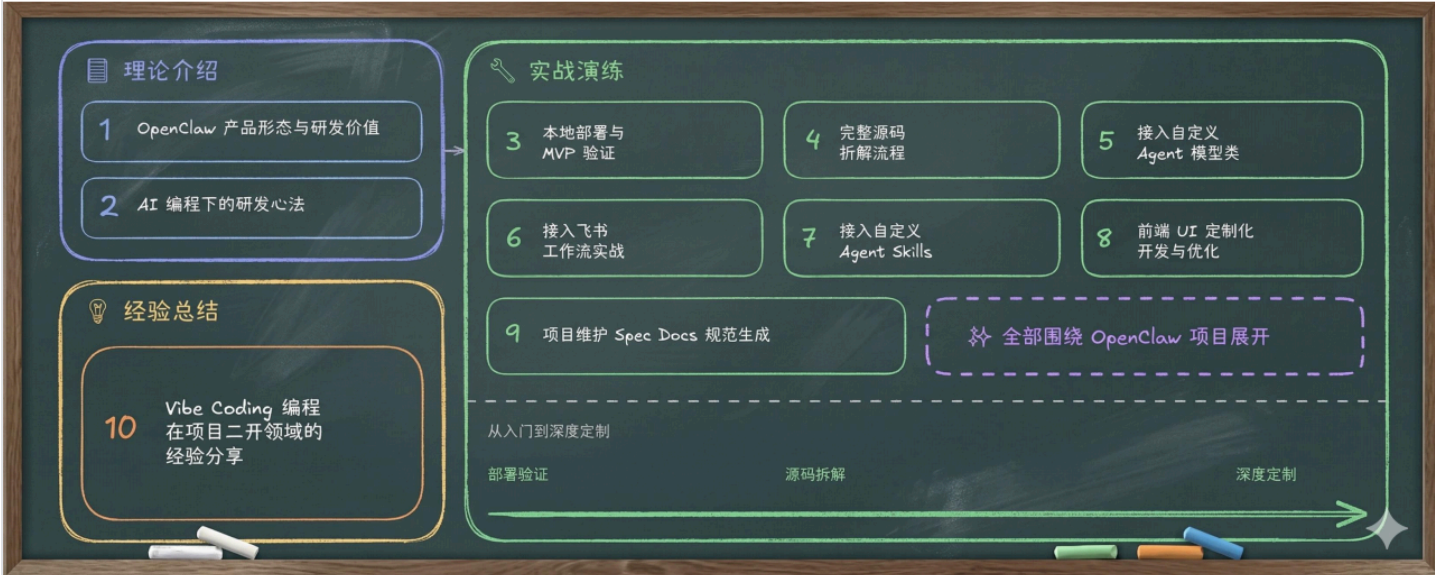


“小龙虾”项目二次开发

“小龙虾”项目二次开发实战



“小龙虾”项目二次开发实战.pdf

1. 下载源码

1.1 本地开发

找个工作目录，然后拉取源码即可。

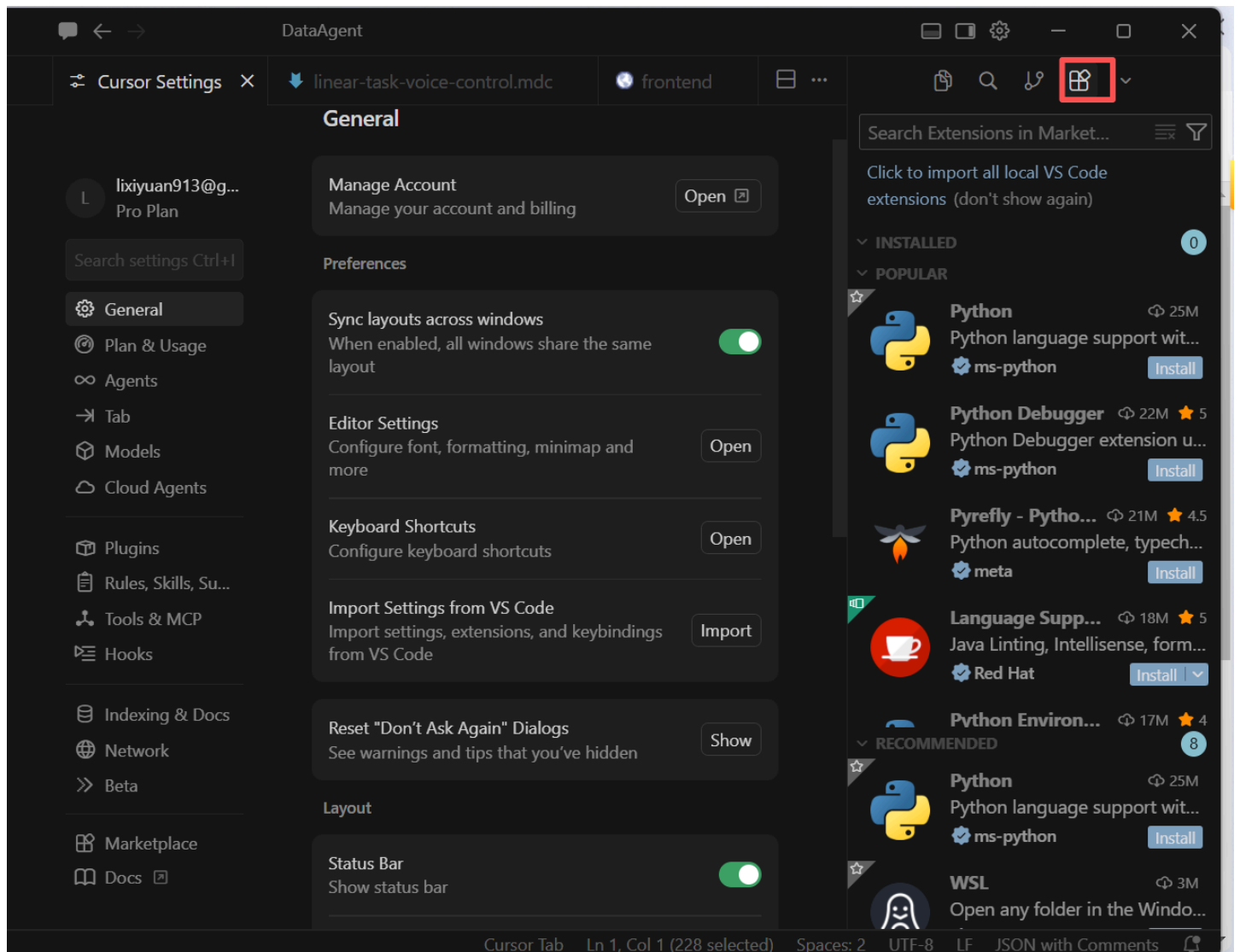
Git 拉去项目：

代码块

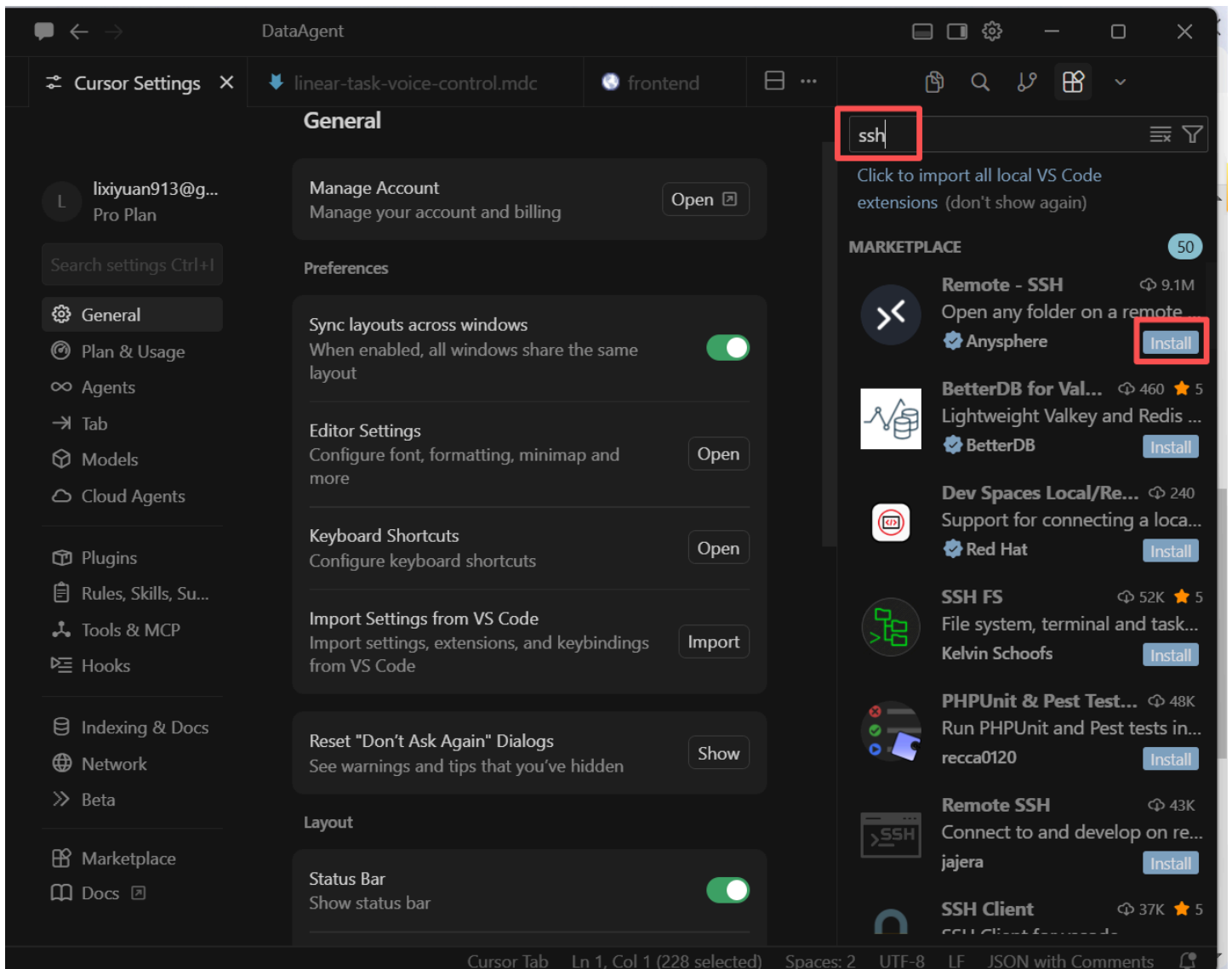
```
1 git clone https://github.com/openclaw/openclaw.git
2 ##ssh ubuntu@106.75.44.248
```

1.2 远程服务器

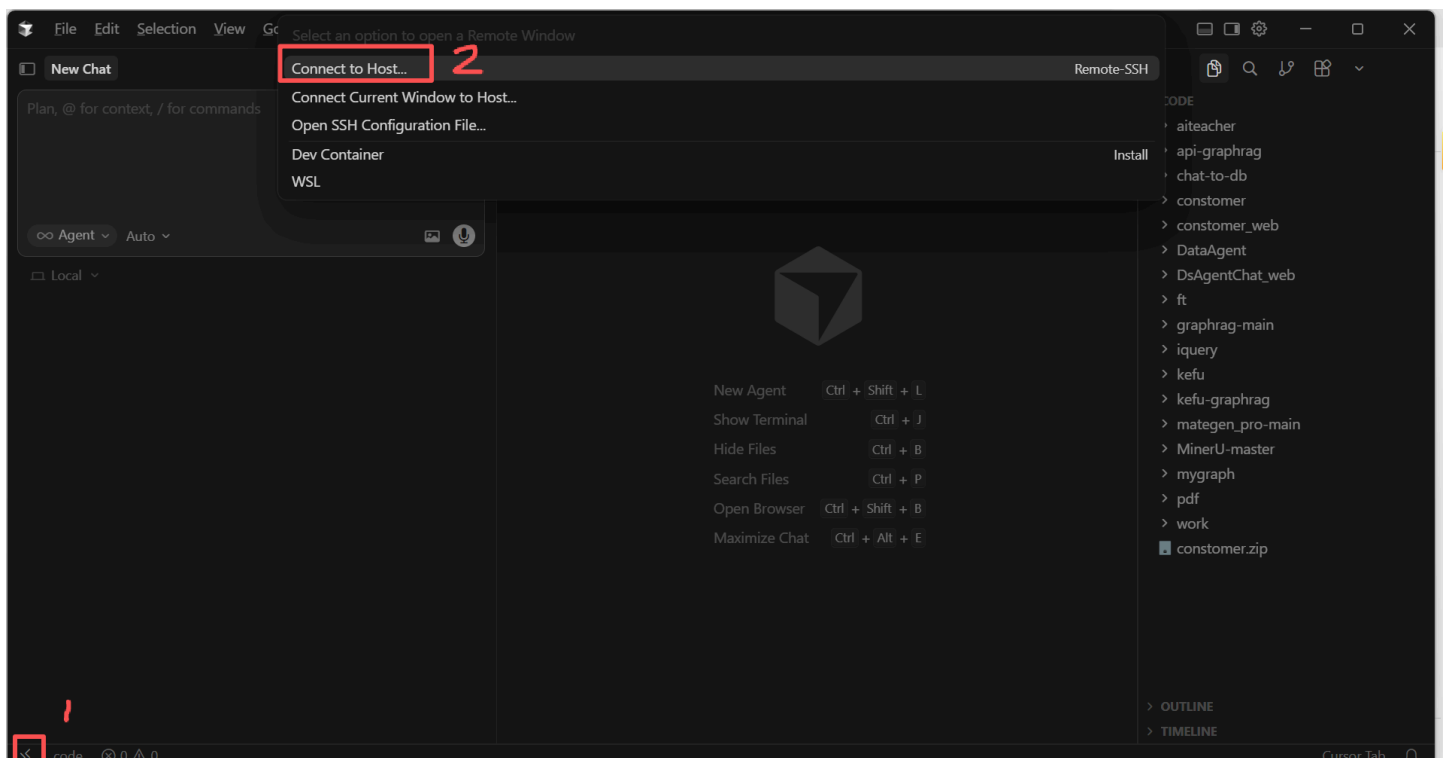
Cursor如何连接远程服务器？

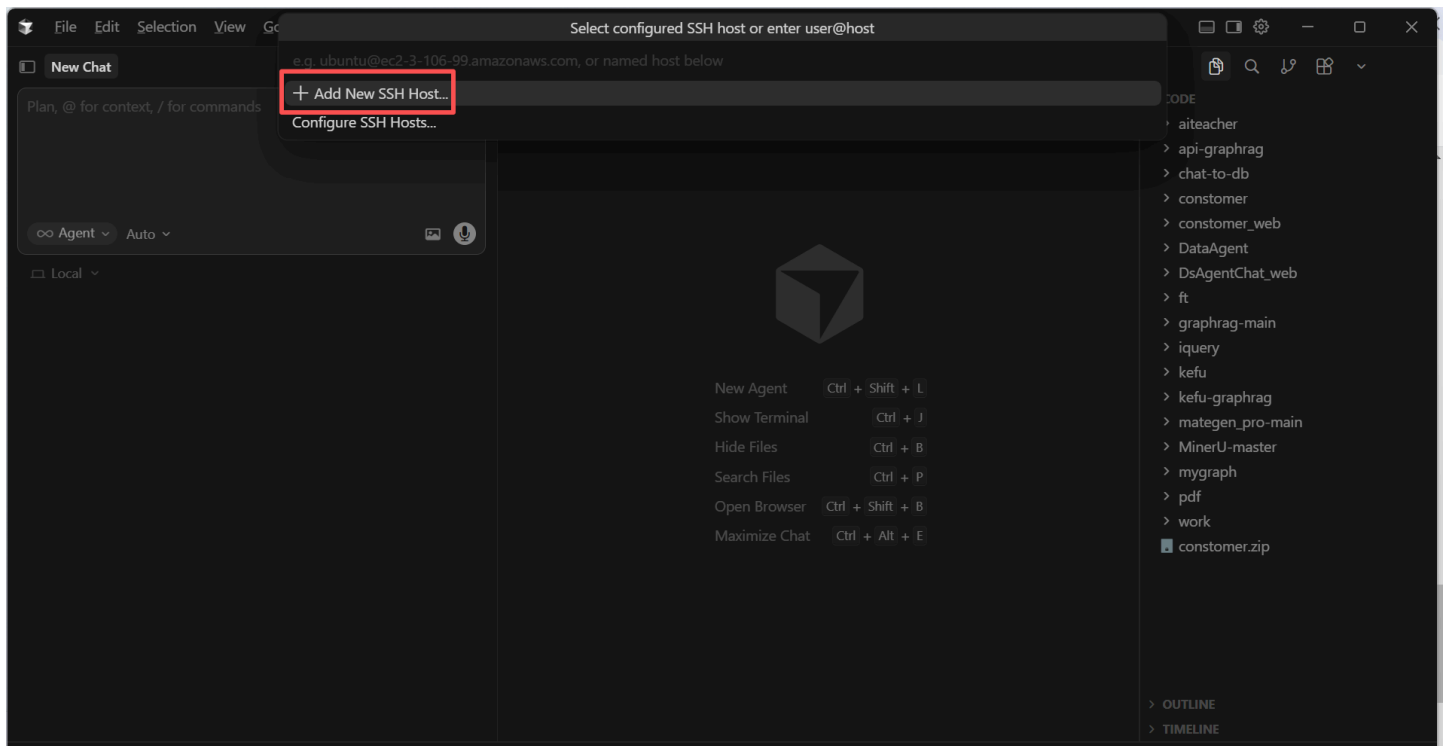


输入 ssh 并 安装 ssh

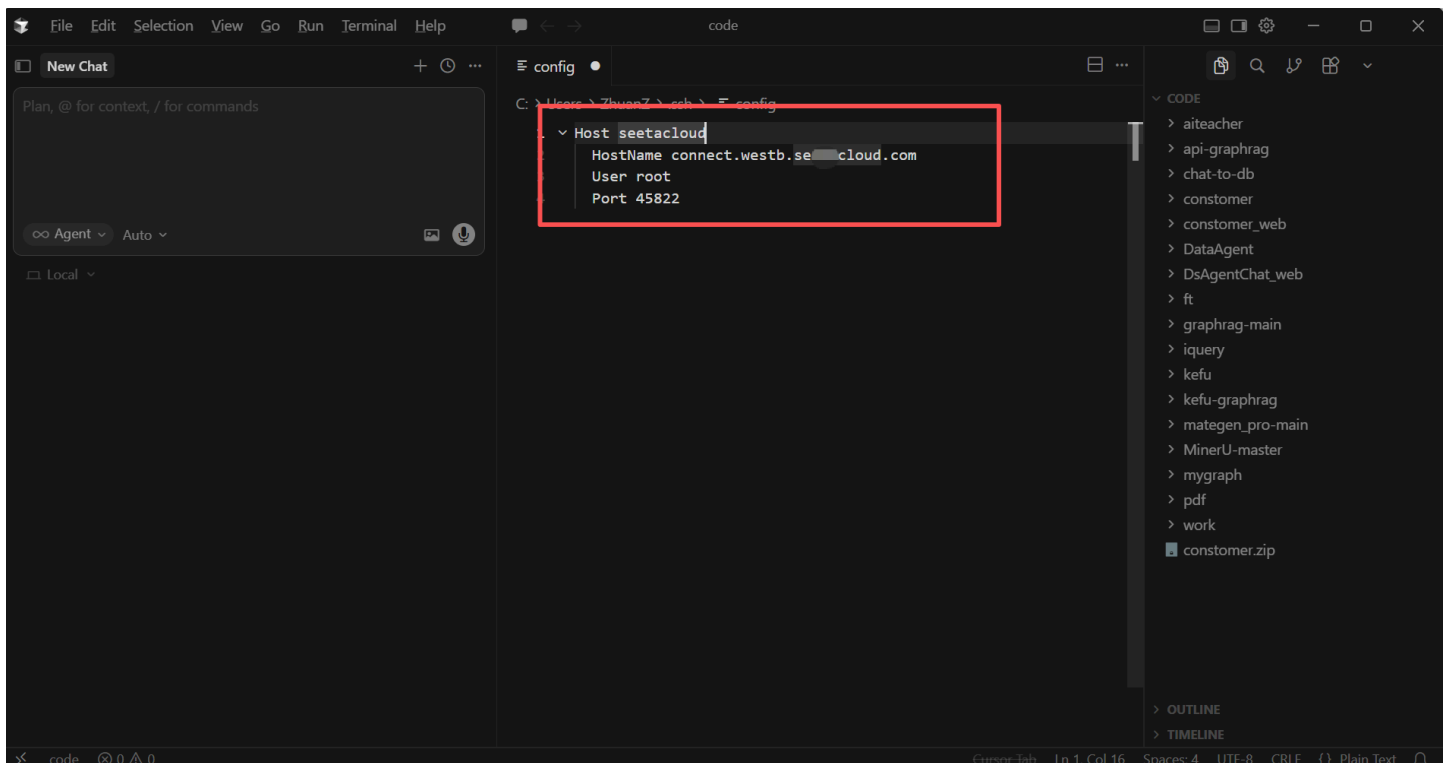


点击 connect to host





添加服务器信息



保存修改后的 `.ssh/config` 文件

点击左下角的绿色远程按钮 → `Connect to Host...`

选择你刚配置的 `seetacloud` （或你自己起的别名）

输入服务器密码或确认密钥，就能成功连接远程服务器了

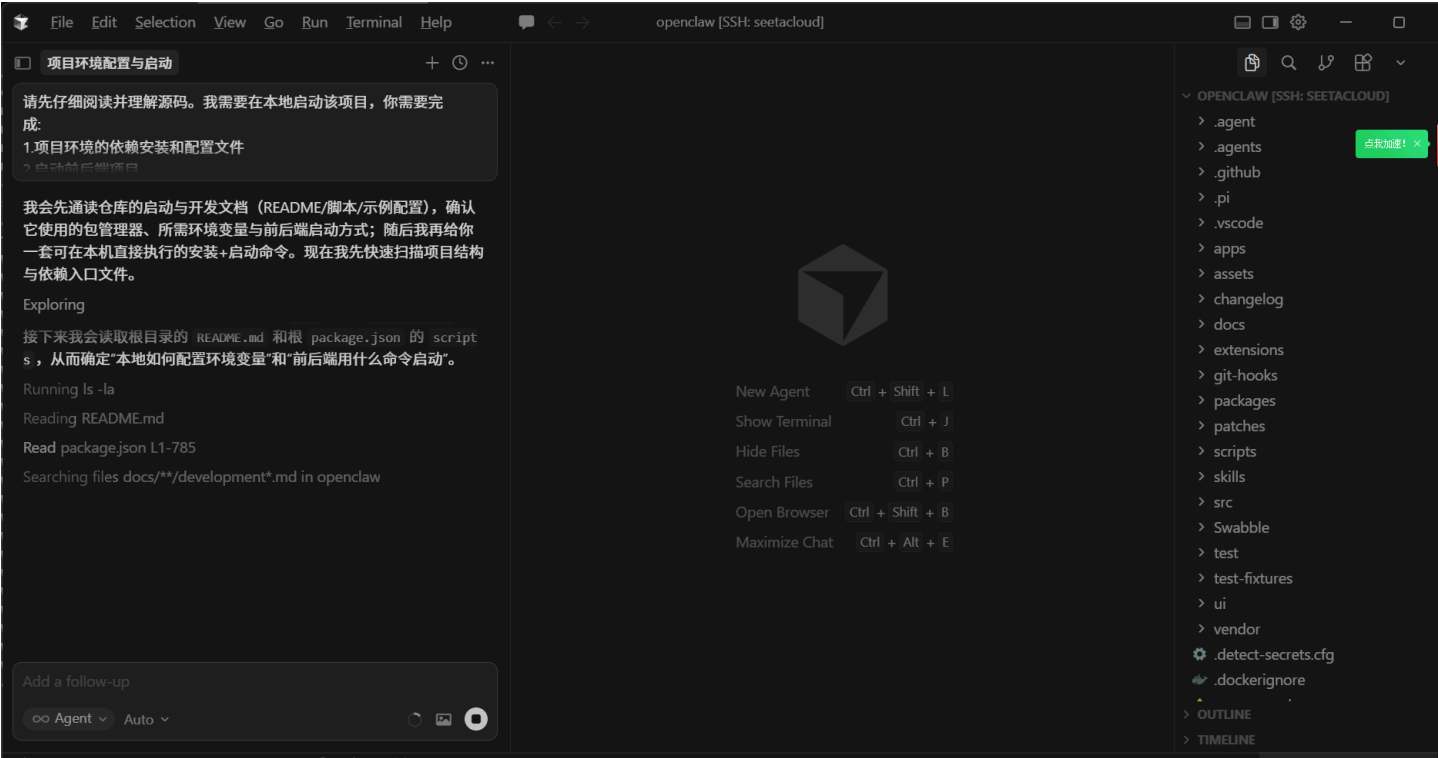
2. 启动源码

2.1 启动项目

代码块

- 1
- 请先仔细阅读并理解源码。我需要在本地启动该项目，你需要完成：
- 2
1. 项目环境的依赖安装和配置文件，需要你就按照，需要配置你就配置
- 3
2. 帮我启动前后端项目
- 4
-
- 5
- #Dashboard URL:

http://127.0.0.1:18789/#token=43e8b6906a52ba6d2eaa2aba0786f11677275627fadc756



启动完了以后，登录的时候可能会遇到token问题，OpenClaw登录的时候需要token的信息。这个可以跟Cursor交互，让他给我们带token的登录地址。

代码块

- 1
- 帮我生成token，然后直接给我带token的登录OpenClaw的地址

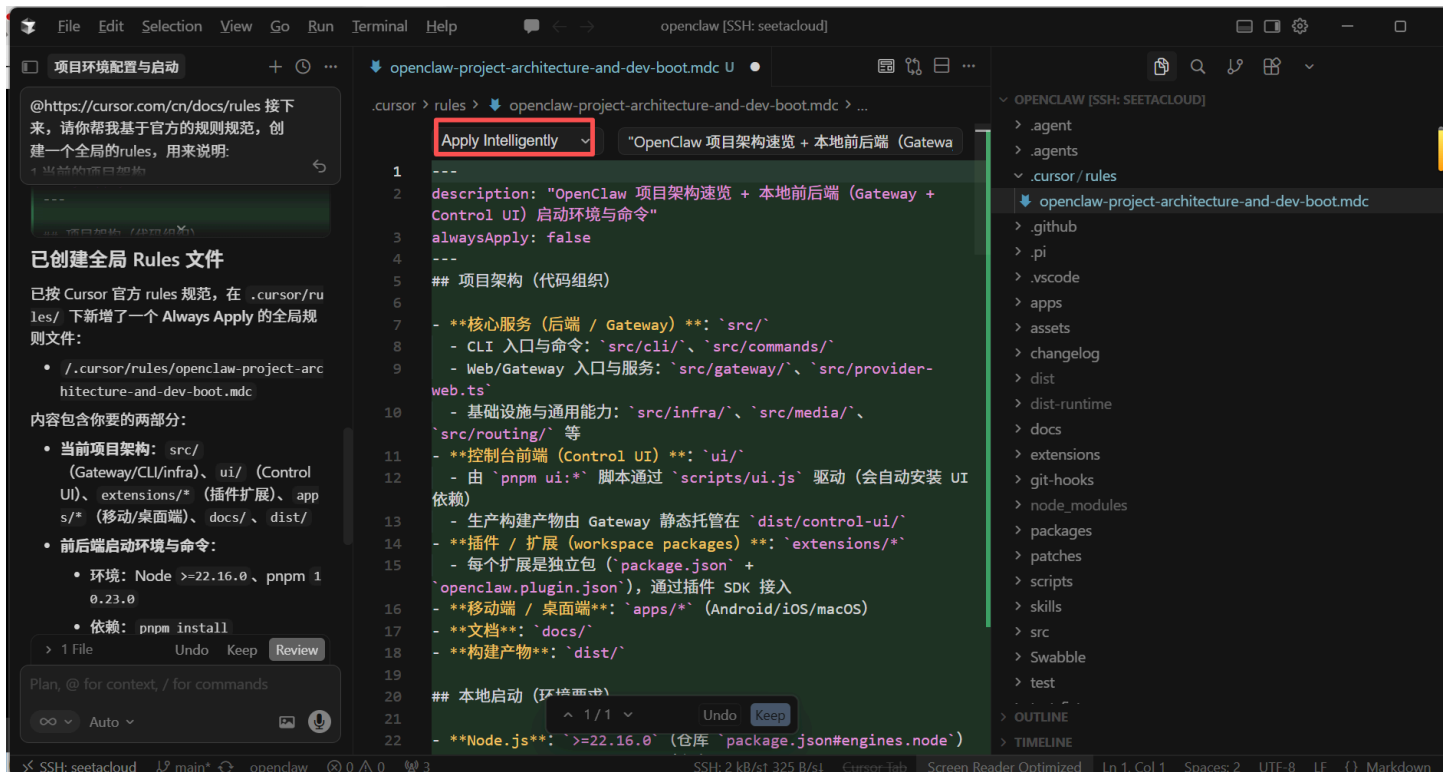
2.2 创建rule

创建rule,我们担心后面Cursor记不起来怎么启动OpenClaw项目，所以我们干脆写一个rule，让他记住这个规则。不要自己写，我们还是借助Cursor编写。

代码块

- 1 @https://docs.trae.ai/ide/rules?_lang=zhTrae 接下来，请你帮我基于官方的规则规范，创建一个全局的rules，用来说明：
- 2 1.当前的项目架构
- 3 2.当前项目的前后端启动的环境以及具体的命令

Cursor写完rule以后，打开看看，如果满足我们的需求，就保留即可，如果rule写得比较复杂可以精简一下，我在准备课件的时候，本次的rule比较复杂，所以我要求Cursor精简一下。



代码块

- 1 我只需要具体的架构说明、前后端服务启动的指令即可，要精简！

3. MVP 测试

3.1 测试对话能力

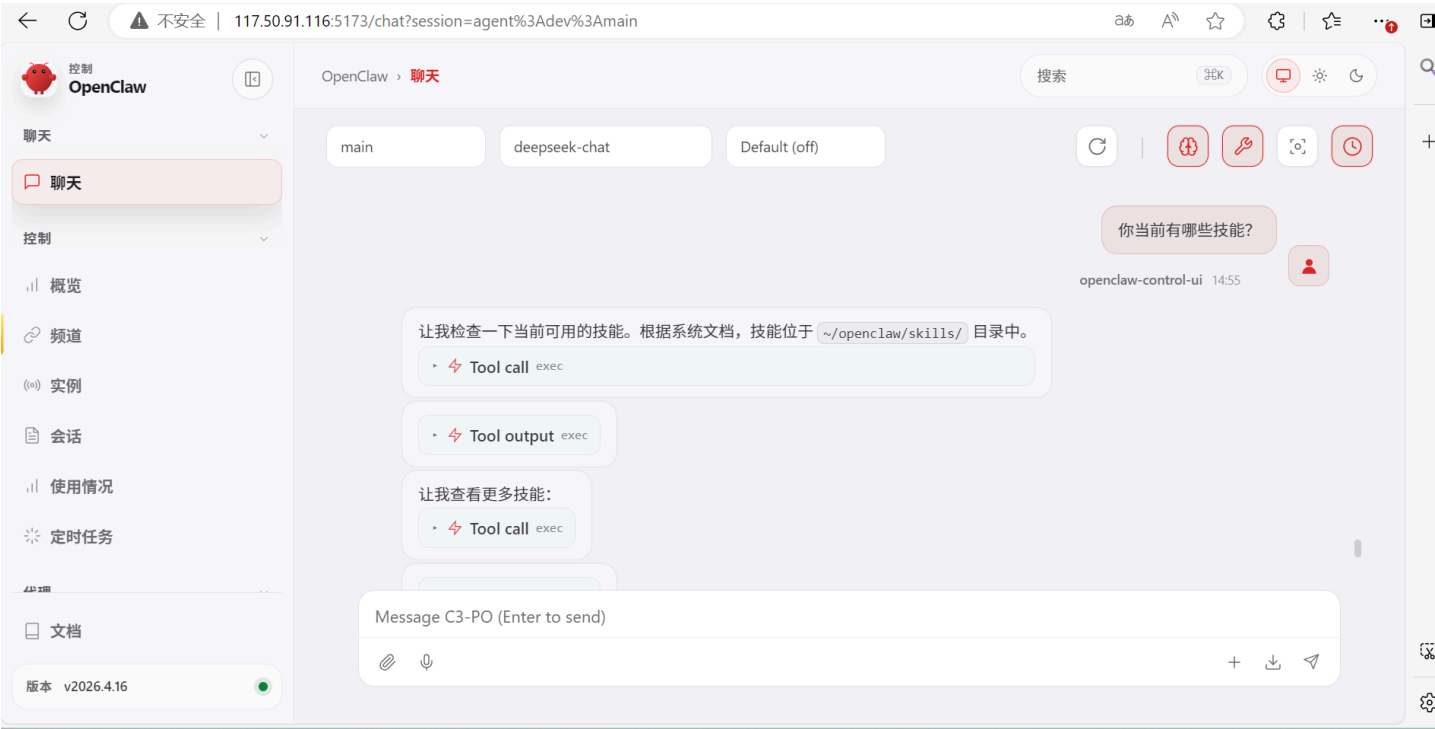
先把简单使用一下小龙虾，我们先配置上接入 DeepSeek 模型。

代码块

- 1 我需要在前端 可以通过 输入mode_name, base_url, key 接入MiniMax 模型，我希望做到的功能是：
- 2 1.在前端左侧的侧边栏单独创建一个Sidebar，用来配置模型
- 3 2.后端需要持久化的保存模型的配置信息，以达到我再次打开首页的时候可以默认加载。
- 4
- 5 # sk-7cec6216ea05406fad6e30f2f9e71b23

6 #
<http://127.0.0.1:18789/#token=43e8b6906a52ba6d2eaa2aba0786f11677275627fadc756>

确定配置完成以后，简单测试一下使用，确保项目没问题。

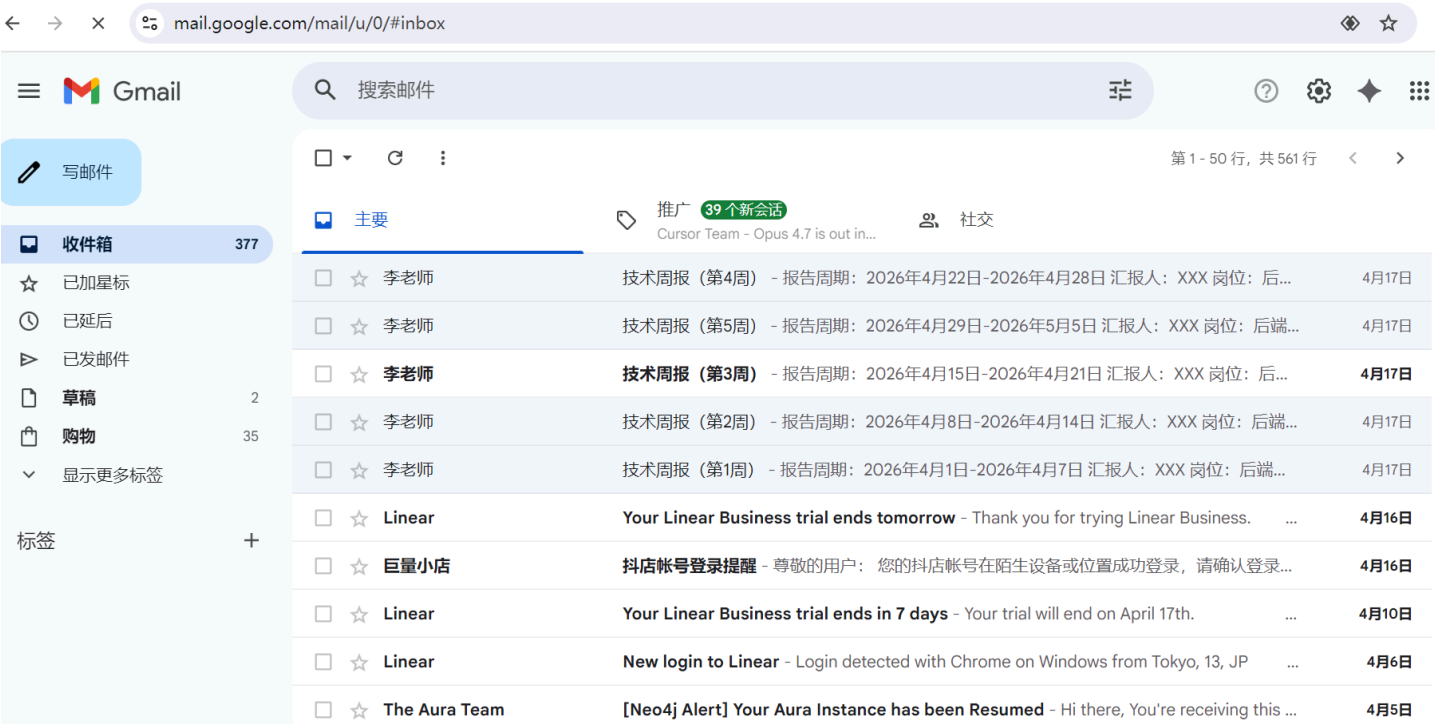


3.2 测试干活能力

让 OpenClaw 登录邮箱读取周报内容并生成年终总结。

我先同步一下我的邮箱情况：我使用的是公司邮箱，访问地址为：

<https://mail.google.com/mail/u/0/#inbox>，登录后效果如下：



接下来，我们让 OpenClaw 实现对邮箱内周报相关邮件的读取，并完成邮件内容的提取与获取。我们先引导 OpenClaw 完成该流程的调试与验证，待功能跑通后，再将这套执行逻辑沉淀为可复用的 Skills。

如下是我们给 OpenClaw 的提示词：

代码块

- 1 接下来，我教你学会通过浏览器操作我的邮箱，然后帮我获取到周报，流程如下：
- 2 1) 首先打开浏览器，我已经登录邮箱，我的邮箱的地址是：
`https://mail.google.com/mail/u/0/#inbox`
- 3 2) 打开邮箱以后，通过搜索邮件功能，输入：周报 两个字，匹配到所有技术周报。
- 4 3) 接下来，你分别读取每个邮件里面的内容，把所有的周报读取到以后进行统一总结，生成年终总结。然后打开通义千问的地址：`https://www.qianwen.com/`，点击里面的 **【PPT创作】** 按钮，然后输入当前的年终总结内容生成PPT
- 5 4) 生成PPT以后，点击右上角下载的按钮，帮我把当前PPT下载到本地的共工作目录。

4. 需求分析

- 接入飞书客户端（也可以是其它客户端）
- 接入自定义SKills
- 接入指定大模型
- 给项目瘦身&优化前端UI

5. 项目结构梳理

5.1 整体项目结构梳理

梳理项目的大的结构

代码块

- 1 接下来，我要进一步探索的是这个项目的整体架构。请你结合源码，帮我详细的分析主要的功能组件以及架构设计，
- 2 我需要系统的了解这个项目的整体设计思路、架构模式以及对应的功能模块，我需要你
- 3 1.结合源码，找到系统模块的划分，明确每个功能模块都是包含哪些代码目录 或核心文件提供服务的；
- 4 2.找到前后端以及外部服务依赖接口的入口文件和各功能模块的集成方式；
- 5 3.你需要梳理出各个服务之间的依赖关系，包括各个模块之间的调用关系，外部的服务集成；
- 6 4.请输出详细的系统架构说明。
- 7 注意:我需要一个整体的架构设计，你仅需要查看项目代码的顶层目录，不需要深入到函数级的分析，同时，明确的标识出顶层结构的功能模块主要是由哪些代码模块提供服务的。
- 8 新建一个文件夹，以md文档的形式写入，注意:请使用中文

把结果生成规范文档（第一个规范文档）

代码块

- 1 新建一个文件夹，以md文档的形式写入，注意：请使用中文

如果Cursor梳理的结果，全部能理解看懂就好，如果有看不懂的，继续追问，确保自己能理解，因为这个过程就是我们借助AI编程工具，快速理解项目的过程。

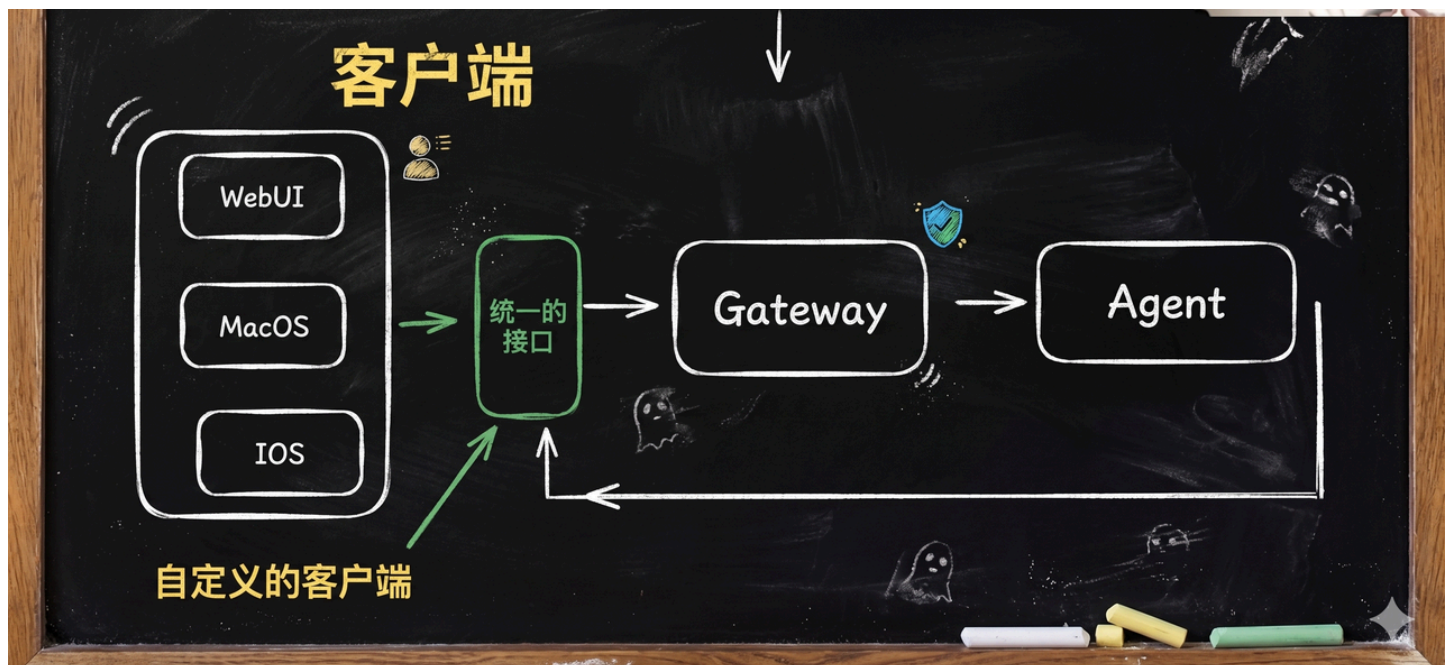
5.2 核心模块梳理-客户端

梳理客户端的架构

提出自己的一些理解，然后去校验自己的想法

代码块

- 1 架构中，客户端层指的是什么呢？举个例子说明一下



代码块

- 1 所以我是否可以这样理解：不同的客户端，例如 Mac、IOS 可以去做一些操控，或者使用一些对应的功能，如消息、摄像头等，对应的数据流会统一通过 GateWay 连接到后端的 Agent 执行用户输入的指

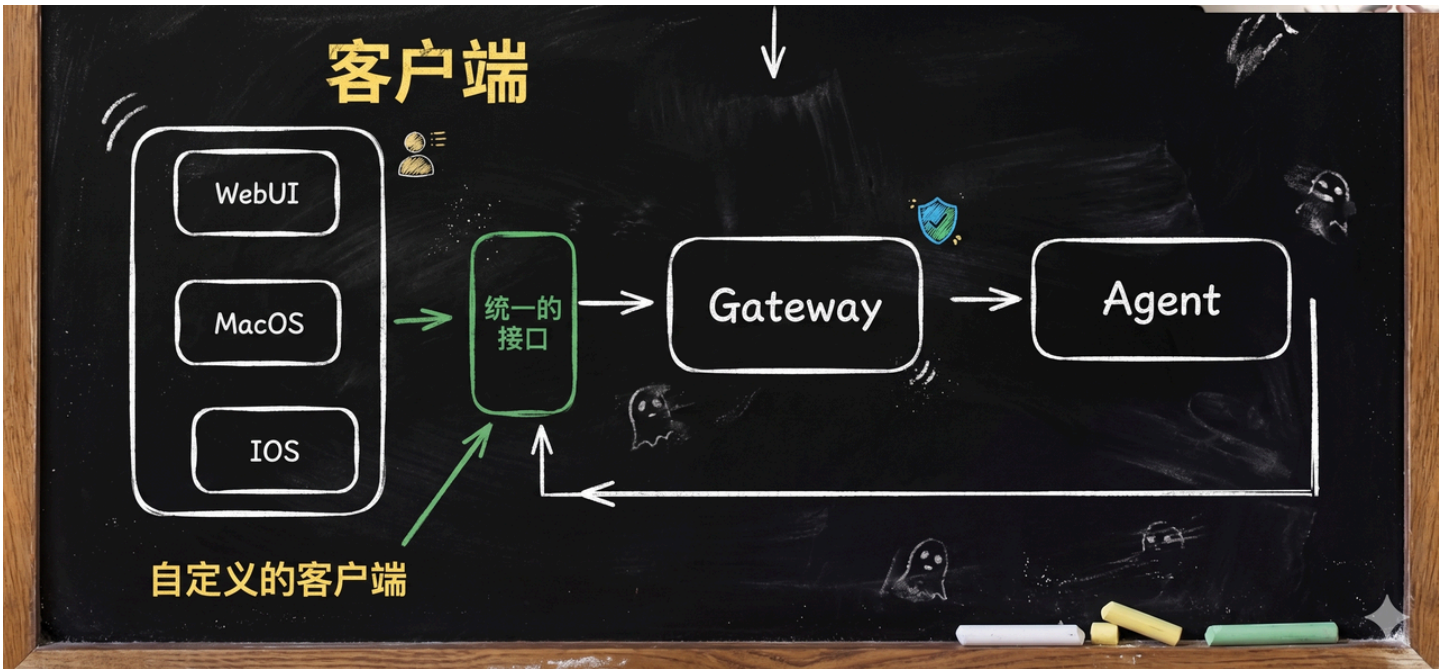
令，并返回到连接的客户端中。

提出自己的疑问

代码块

- 1 客户端和我启动的UI界面之间的关系是什么？ UI通过网页端操作，而客户端是不
- 2 是指的不使用网页端，而是接入外部的设备？

基本确认大的架构



我们后面要进行二次开发，所以我们现在可以把客户端部分生成规范文档。

整理成规范文档（第二个文档规范）

代码块

- 1 我现在已经梳理清楚了，该项目的第一层就是客户端，主要是包含了WebUI、MacOS等不同的客户端，我现在需要你详细的解析一下源码，从如下几个点入手：
- 2 1. 他们都是通过相同的输出与GateWay进行通信，那么不同的客户端，在构建过程中都有哪些相同、以及异同，请详细展开记录
- 3 2. 如果我要接入不同的客户端进行二次开发，都需要做什么配置，以及需要从哪些功能点入手？
- 4 请结合源码，详细的分析并输出一份.md文档，命名为phase1_xxx.md（你需要重新命名）

5.3 核心模块梳理-GateWay

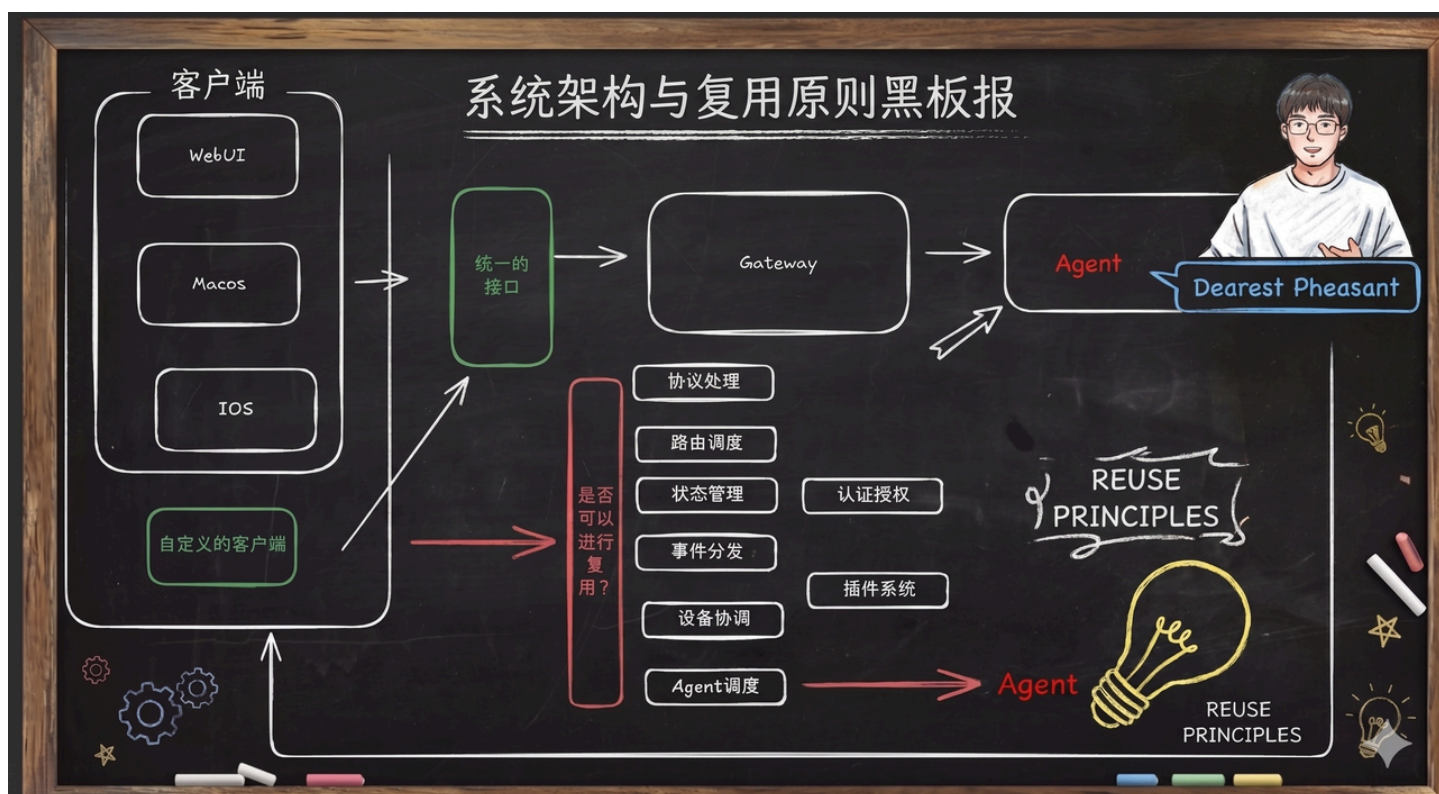
接下来了解Gateway的架构

代码块

1 OpenClaw里面的Gateway核心的功能是什么？

代码块

- 1 接下来，我需要详细的分析一下GateWay组件，这个组件的输入和输出，更像是接收不同的客户端，然后给到后端的Agent进行执行，是这样的处理逻辑吗？



我们简单了解了Gateway的功能以后，我们接下来要关注一个核心的问题，就是我们二次开发接入新的客户端的时候，我们需不需要动GateWay？

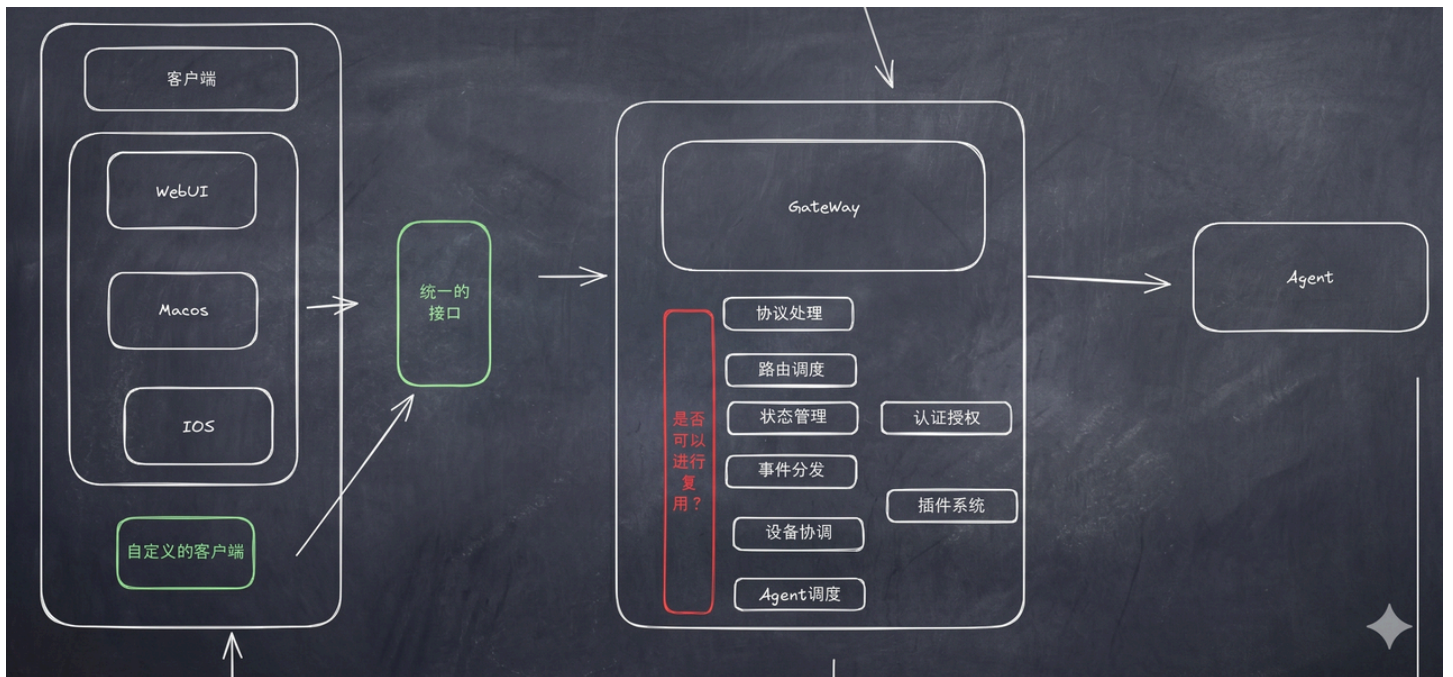
代码块

- 1 我现在的理解是这样的：GateWay是一个比较完整的后端系统，但是划分了不同的模块，我理解的是：像协议处理、路由调度、状态管理、Agent调度等核心机制，理论上，如果我接入不同的客户端，那么现在项目的源码处理逻辑，是否已经完成了对接？换句话说：这一部分的核心代码我能不能百分之百的进行复用。请结合源码仔细分析

生成规范文档

代码块

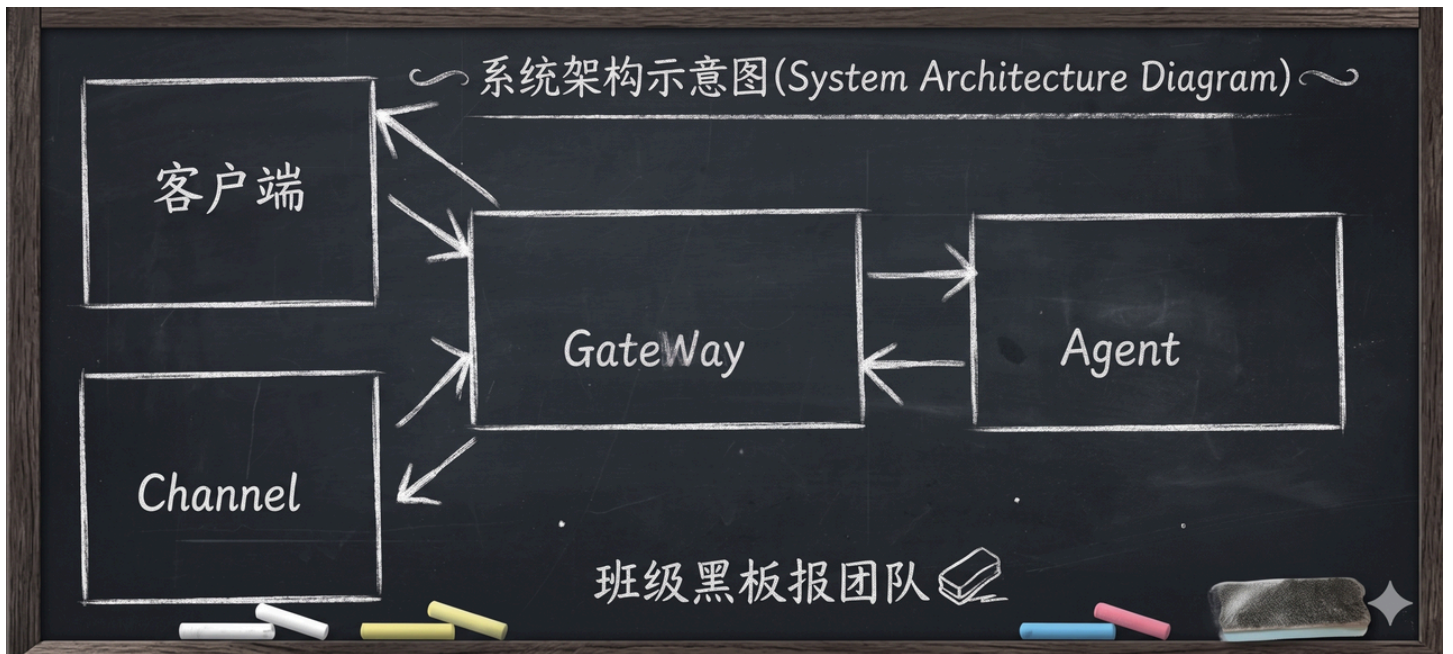
- 1 前面我已经梳理清楚了GateWay的核心组件，其中GateWay是完整的后端系统，但是划分了不同的模块，所以我需要你做的事情是：
- 2 1.只要我通过WebUI、Macos等客户端接入。理论上GateWay是可以百分之百复用的，所以我需要你结合源码，详细的梳理出当前架构中代码的标准接入方式以及核心功能模块
- 3 2.需要结合源码，确定不同的客户端在GateWay对接时，采用的输入和输出的接口规范
- 4 3.如果我要二次开发新的客户端连接，需要关注哪些技术细节
- 5 请结合源码，详细的分析并输出一份.md文档，命名为phase2_xxx.md



5.4 核心模块梳理-Channel

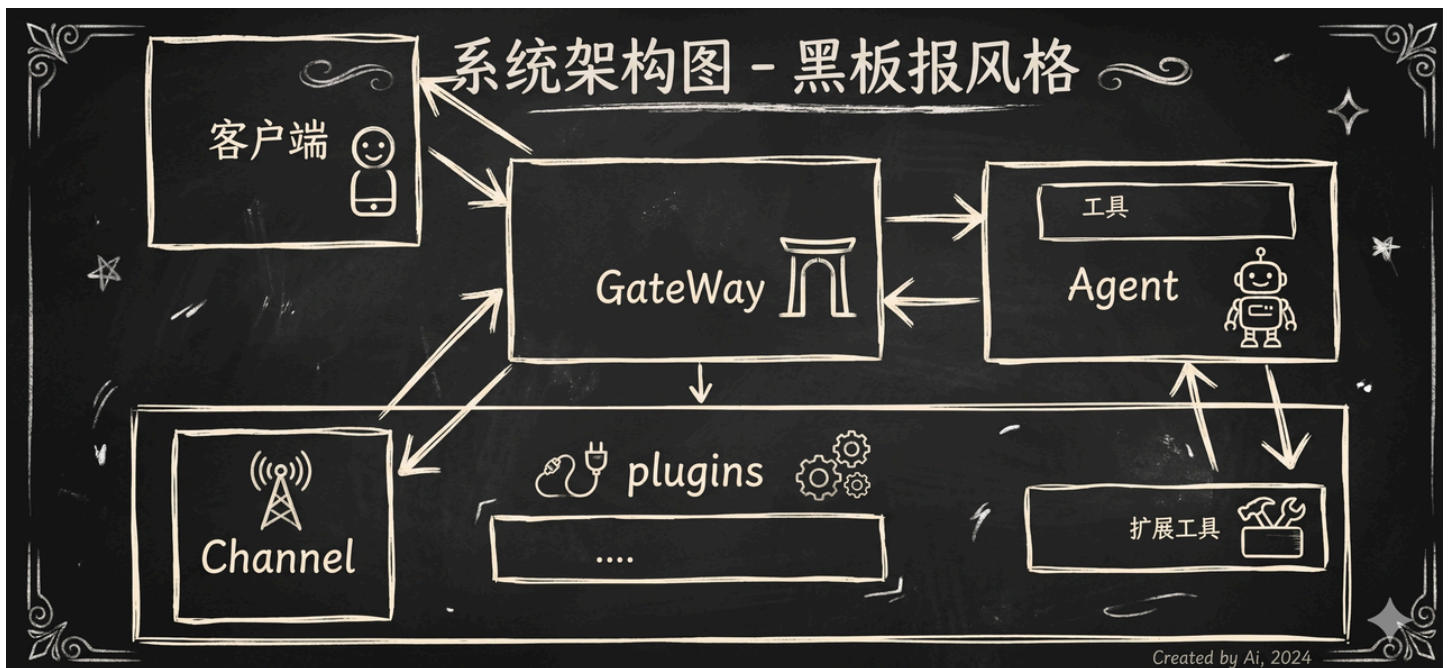
代码块

- 1 ok，接下来我需要继续进行拆分，你提到了 客户端，GateWay、渠道 他们之间的关系是什么？



代码块

- 1 ok, 接下来我需要进行拆分, 你提到了GateWay, Agent、channels和Plugins, 这四个概念, 他们之间的关系是什么?

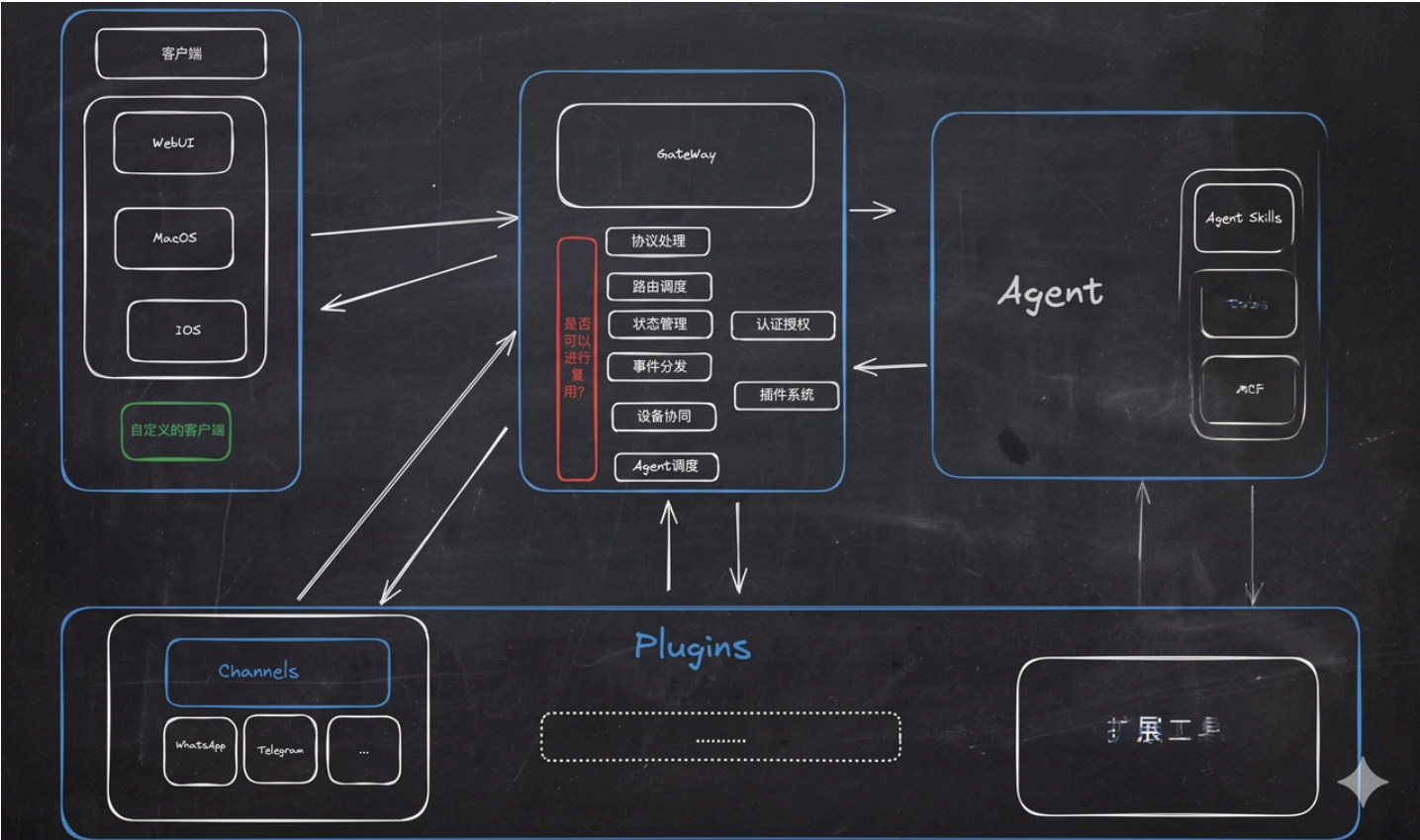


代码块

- 1 我现在有一个疑问:Channel 是否可以随着接入的不同服务而有所不同, 比如我接入telegram 和 whatsapp,所用的Channel是一样的吗?请结合源码帮我回答.

我之所以要问这样的一个问题是因为我想接入飞书，我是改Plugins，还是改Channel？

如果它告诉我用的都是不同的Channels，那么我接入飞书的时候我得自己单独设计一个Channel？



答案是需要一个独立的channel

代码块

1 现在我已经明白了。不同的channel，是否有相同的输入和输出接口。我现在能理解的是：每一个不同的channel，内部执行的流程可以不同，但是既然都可以接入到相同的Agent架构中，以及Agent都是通过同一个GateWay接入，那么肯定是用相同的输入和输出。所以请结合源码，明确的告诉我，是否存在这样一种规范？

答案是肯定的！

生成文档规范（第四个文档规范）

代码块

1 我后续需要接入自定义的channel，所以我需要你做如下几件事情：

2 1. 找到Channels统一的输入输出接口规范，你需要详细的帮我把这个接口规范写清楚，从而能指引我后续接入自定义的channel后，正确的对接到当前的系统中；

- 3 2. 请结合源码，详细的分析并输出一份.md文档，命名为phase3_XXXX.md(注意:xxx是占位符，你要命名文件)

5.6 核心模块梳理-Agent

拆分Agent

代码块

- 1 接下来，我需要重点的探索Agent 的内部架构了，对于Agent Runtime，我能想到是：
- 2 1.底层应用的Agent技术框架是什么
- 3 2.大模型接入的方式是什么。支持哪些类型的模型接入？
- 4 3.大模型是如何接入Agent Skills、MCP 或者 Tools？
- 5 4. Agent的记忆系统是如何设计的？
- 6 5. Agent的Session会管理是如何设计的？
- 7 这是我目前能想到的通用 Agent 的基础架构模块，请你结合项目源中 Agent 构建的核心类方法，详细分析一下是否有遗漏，以及源码中对应的每个模块都是如何实现的？
- 8





生成文档规范（第五个文档规范）

代码块

1

接下来，我要去拆分Agent运行时的架构设计。Agent主要包含：

2

1.底层框架:基于pi-agent-core，提供完整的Agent运行时

3

2.模型支持:支持OAuth、Failover、Rate Limit 处理

4

3.工具系统；

5

4.记忆系统:短期(Session Transcript)+长期(LanceDB 向量记忆)，支持自动召回和捕获

6

5.会话管理:基于 sessionKey 的隔离机制，支持私聊/群组/全局模式，完整的生命周期管理

7

你需要围绕这几项，结合源码进行详细的分析和解读，请结合源码，详细的分析并输出一份.md文档，命名为phase4_XXXX.md(注意:xxx是占位符，你要命名文件)

5.7 校验文档规范

代码块

1

[phase1_cli_spec.md](architecture-overview/phase1_cli_spec.md)
[phase2_gateway_reuse_contracts.md](architecture-overview/phase2_gateway_reuse_contracts.md) [phase3_channel_plugin_io_spec.md](architecture-overview/phase3_channel_plugin_io_spec.md)
[phase4_agent_runtime_architecture.md](architecture-overview/phase4_agent_runtime_architecture.md) 这是我拆分出来的四个架构模块，请你最后再 全局的阅读源码，看下是否还有遗留的核心架构！

6. 版本升级

目前我已经把规范文档梳理完毕，按正常流程推进开发即可。不过我想再模拟一个实际场景：在我们做二次开发时，选用的开源项目热度较高，就在我们生成规范的这几天里，源码又进行了重要升级，而我们又希望基于这个最新版本开展开发。这种情况下，我们应该如何处理？

6.1 重新下载源码

下载最新的源码

代码块

```
1 git clone https://github.com/openclaw/openclaw.git
```

建议，下载到新的源码以后，再做一下MVP的测试。

6.2 更新开发规范

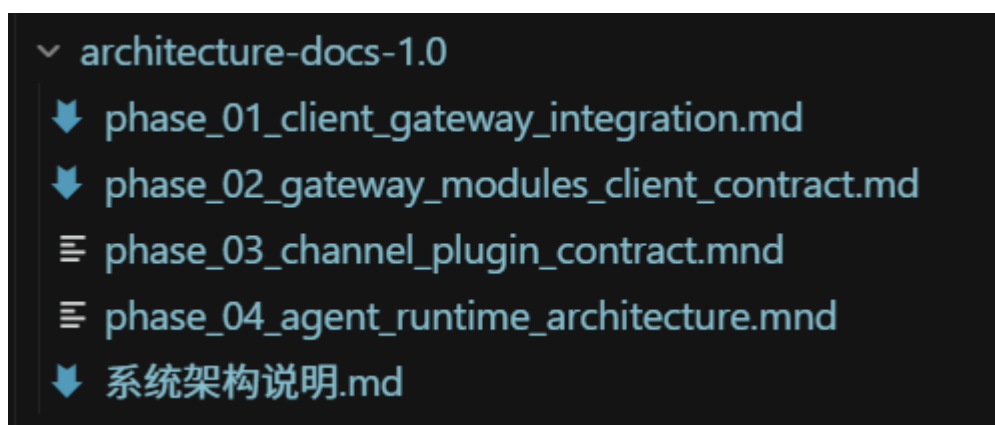
帮我们之前整理的文档规范复制到新的项目里。

基于新的项目更新当前的文档

代码块

```
1 @openclaw/architecture-docs-1.0/系统架构说明.md 这是我之前梳理的OpenClaw的系统架构说明文档，现在OpenClaw已经更新了多个版本，我需要你系统的阅读并理解最新版本的代码及系统架构，对改动以及新增的功能进行详细的梳理，最终帮我更新 OpenClaw系统架构说明.md 并输出为 OpenClaw系统架构说明v1.0.md
```

接下来，陆续也对 4个文档规范进行更新。



代码块

```
1 phase1_client_gateway_integration_guide.md 这是我之前梳理的OpenClaw的客户端模块开发，现在
2 OpenClaw已经更新了多个版本，我需要你系统的阅读并理解最新版本的代码及系统架
3 构，对改动以及新增的功能进行详细的梳理，最终帮我更新
  phase1_client_gateway_integration_guide.md.md并输
```

4 出为 phase1_client_gateway_integration_guide.md-v1.0.md

代码块

```
1 phase2_gateway_standard_integration_and_client_io.md这是我之前梳理的OpenClaw的客户
  GateWay模块
2 开发规范，现在OpenClaw已经更新了多个版本，我需要你系统的阅读并理解最新版本
3 的代码及系统架构，对改动以及新增的功能进行详细的梳理，最终帮我更新
4 phase2_gateway_standard_integration_and_client_io.md 并输出为
5 phase2_gateway_standard_integration_and_client_io-v1.0.md
```

代码块

```
1 phase3_custom_channel_plugin_contract_and_io.md 这是我之前梳理的
2 Openclaw的Plugins和 Channel开发规范，现在
3 Openclaw已经更新了多个版本，我需要你系统的阅读并
4 理解最新版本的代码及系统架构，对改动以及新增的功能
5 进行详细的梳理，最终帮我更新
6 phase3_custom_channel_plugin_contract_and_io.md 并输出为
7 phase3_custom_channel_plugin_contract_and_io_v1.0.md
```

代码块

```
1 phase4_agent_runtime_architecture.md 这是我之前梳
2 理的Openclaw的AgentRuntime开发规范，现在
3 Openclaw已经更新了多个版本，我需要你系统的阅读并理
4 解最新版本的代码及系统架构，对改动以及新增的功能进
5 行详细的梳理，最终帮我更新
6 phase4_agent_runtime_architecture.md 并输出为
7 phase4_agent_runtime_architecture_v1.0.md
```

7. 二次开发

7.1 项目启动

代码块

```
1 帮我启动一下当前项目，如果需要安装依赖，请先安装依赖，然后最后帮我生成带token的登录链接。
```

7.2 接入自定义Model

代码块

- 1 我现在要进行项目的二次开发，首先我需要确定的是：我想要接入DeepSeek的 deepseek-v4-pro 模型， 你结合@phase4_agent_runtime_architecture_v1.0.md 看一下有什么方案？

代码块

- 1 现在我选择使用接入方式：用C方案进行接入，接入DeepSeek的deepseek-v4-pro模型
- 2 我不想通过控制台的后台终端配置，我希望能够在 WebUI 页面上填写， Provider,baseUrl, model_name,API_key 所以我希望你做的事情是：
- 3 1. 开发前端的WebUI，左侧的侧边栏新增一个支持在线配置模型的页面。同时，配置成功后可以持久化的存储和写入到后端的文件中。这样我以后再次打开系统的时候可以默认加载到已经配置过的模型。
- 4 2. 我需要将配置的deepseek-v4-pro模型作为 Agent Runtime的默认模型。

测试key: sk-ea71f1088b0542c6a55f9e6aac522692

base_url: <https://api.deepseek.com>

model_name: deepseek-v4-pro

7.3 接入自定义Channel

代码块

- 1 @phase3_custom_channel_plugin_contract_and_io_v1.0.md

代码块

- 1 好的，我现在的思路是这样的：
- 2 我选用方案A
- 3 1. 新增一个 前端 UI 的 Channel， 用来做feishu 的 在线配置，你需要将 用于连接 feishu 的 App ID 和 App Secret 等必要的配置信息，结合源码的接口一一进行匹配。
- 4
- 5 2. 移除现有的 Channel UI中的其余的Channel，仅保留 Feishu 即可。

我的App ID 是: cli_a967a33bcbb89bc4

我的App Secret 是: suE6FOCoHb02aw2PkMFPLby0IXueAQm4

飞书插件安装与应用接入

进入飞书开发者控制台

打开飞书开发者控制台：<https://open.feishu.cn>，按页面提示创建并初始化应用。



先进入开发者后台首页，点击创建企业自建应用，进入应用初始化流程。



创建企业自建应用

该应用仅可在“星河探索”内部使用，应用发布需经过企业管理员审核，请仔细阅读 [应用审核规则](#)。

应用名称*

OpenClaw8/32

应用描述*

我的小龙虾5/120

应用图标*

JPEG/PNG/SVG/BMP 格式，2 MB 以内，大于 240*240 px，无圆角

选择背景色

取消

创建

在应用基础信息页补全应用名称、描述和可见范围，保存后进入下一步配置。

创建机器人

进入机器人能力配置入口，先启用机器人功能。

飞书开放平台

客户案例应用中心开发文档智能助手 AI 开发

你可以输入文档关键词、开发问题、Log ID、错误码

开发者后台

OpenClaw 待上线

正式应用@星河探索

应用发布后，当前配置方可生效

创建版本

添加应用能力

1. 你可以根据实际需求，为应用开启能力。单个应用可开启多种能力，一个能力可用于一个或多个飞书场景。

2. 如果尚不确定需要添加什么能力，可切换至“按场景添加”，根据所需的业务场景选择能力。[了解更多](#)

按能力添加

按场景添加 new

机器人

与用户在聊天中交互的应用，它可以向用户或群组自动发送消息，响应用户的消...

+ 添加

网页应用

快速接入已有的网页应用，用户可以通过飞书客户端免登录快速进入。

+ 添加

工作台小组件

将数据图表、图文资讯等信息通过小组件添加到工作台。

+ 添加

云文档小组件 可添加多个

在云文档中扩展内容或者添加快捷操作。

+ 添加

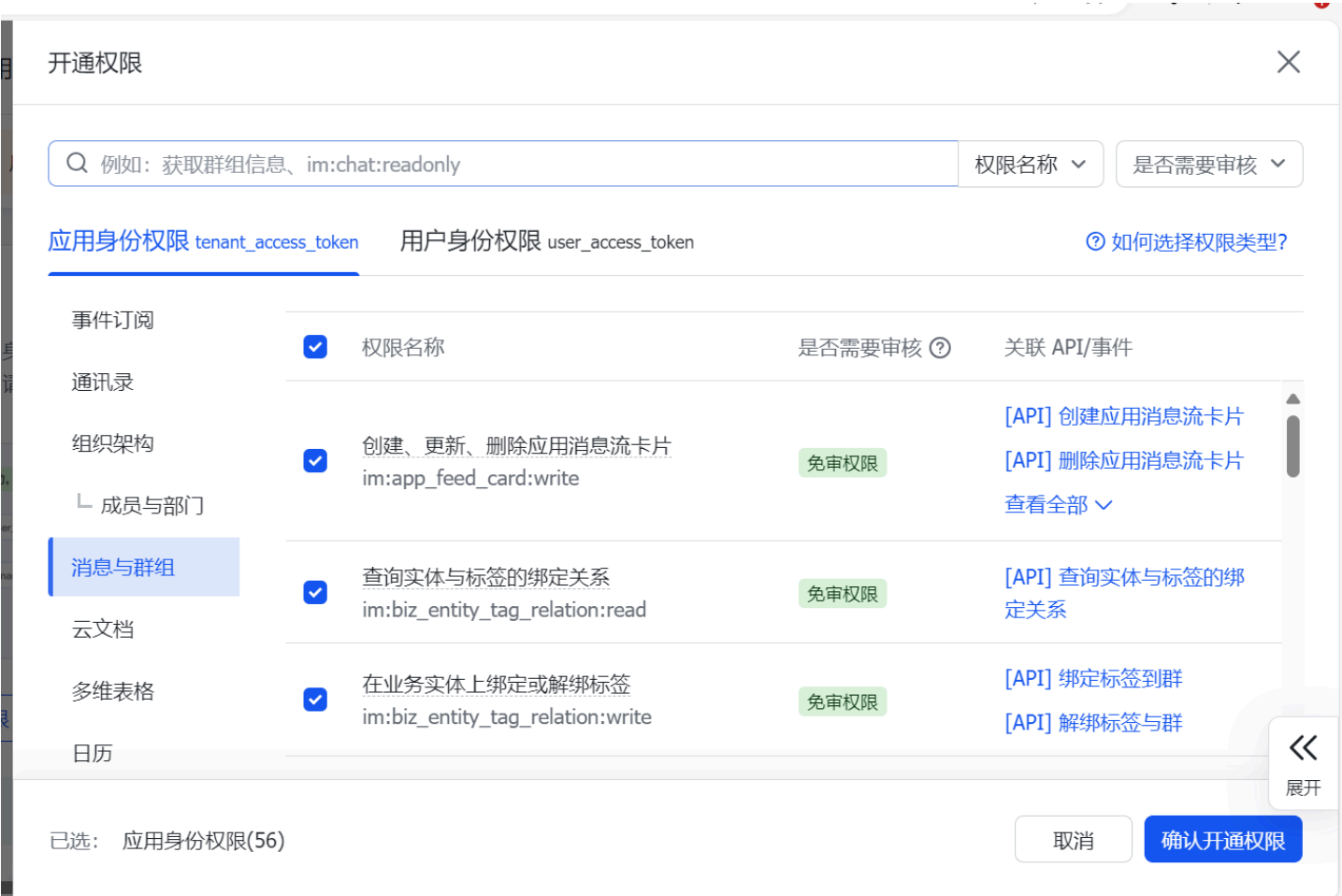
配置机器人头像、名称和简介，确保群成员能明确识别这是业务机器人。

机器人权限配置

进入权限管理页，按截图逐项勾选必需权限。权限缺失是消息无法收发、群事件无法触发的最常见原因。



主要开通接收消息



事件订阅

- 事件与回调 → 订阅方式：选择 “使用长连接接收事件/回调”

事件配置

订阅事件后，飞书开放平台将会在事件（如机器人入群）发生时向请求地址推送消息 [了解更多](#)

订阅方式 [🔗](#)

使用 **长连接** 接收事件 **推荐**

配置方式

无需注册公网域名或配置加密策略，仅需使用 [官方 SDK](#) 启动长连接飞书客户端，并确保连接成功后，即可开启该模式。
配置完成后，可点击按钮验证连接状态

- 事件订阅：确保已勾选/添加 `im.message.receive_v1`

已添加事件

添加事件

事件名称	订阅类型 ②	所需权限（开通以下任一权限即可）	操作
接收消息 v2.0 im.message.receive_v1	应用身份	读取用户发给机器人的单聊消息 已开通 获取群组中其他机器人和用户@当前机器人的消息 已开通 获取群组中用户@机器人消息 已开通 获取群组中所有消息（敏感权限） 已开通	删除事件

版本管理与发布

飞书开放平台 客户案例 应用中心 开发文档 智能助手 AI 开发

你可以输入文档关键词、开发问题、Log ID、错误码

开发者后台

OpenClaw 待上线 正式应用@星河探索

应用发布后，当前配置方可生效 创建版本

版本管理与发布

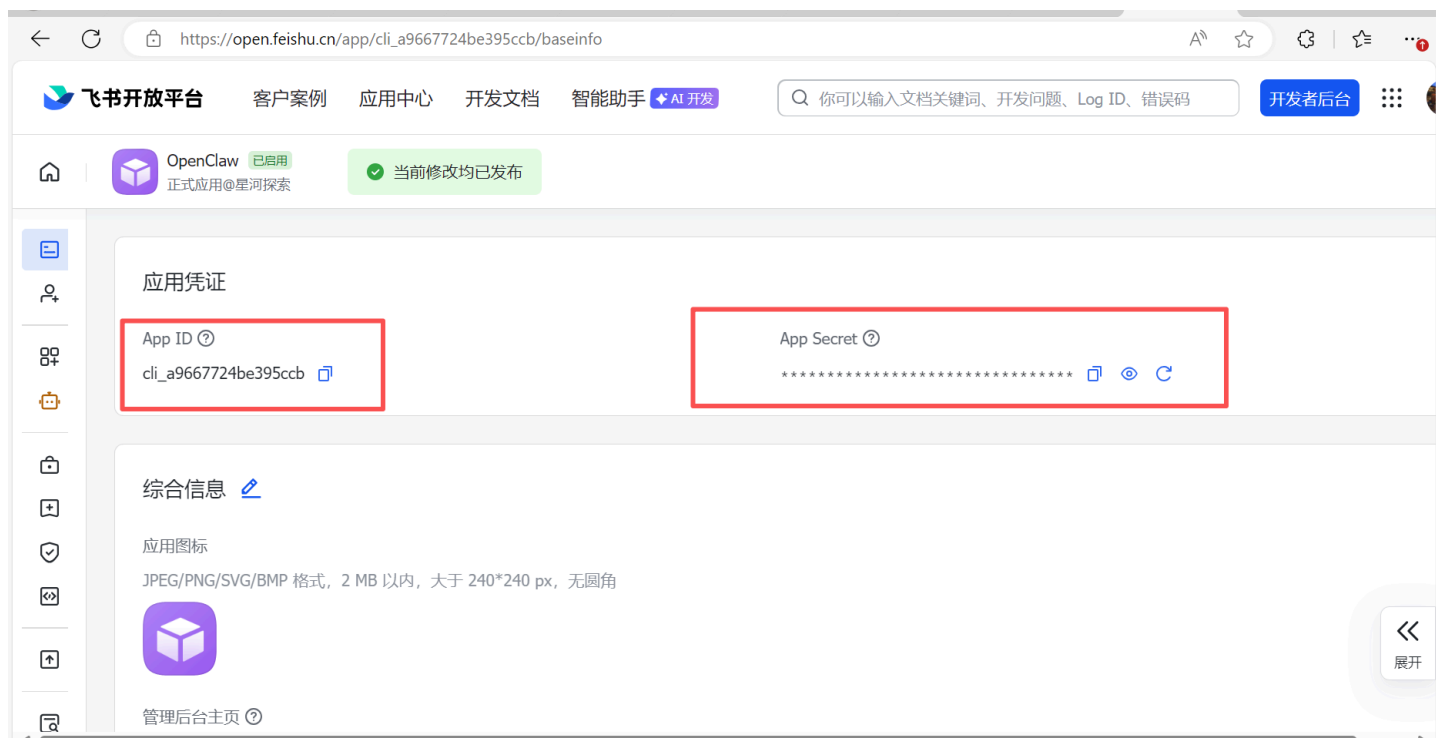
创建版本

若企业自建应用需要供企业内其他用户使用，或商店应用需要上架飞书应用中心，均需先发布版本

版本号	版本状态	发布人	发布通过时间	操作
暂无数据				



获取到应用凭证



7.4 接入自定义Skills

<https://clawhub.ai/>

代码块

- 1 @phase3_custom_channel_plugin_contract_and_io_v1.0.md 请结合这个文档+实际的源码，帮我梳理下 如果新建一个 channel 并且需要加载 自定义的 skills，应该遵循什么规范？

代码块

- 1 ok，现在我需要在 feishu 的 channel 中 除了使用内置的 skills，帮我按照相同的规范创建一个PPT Skills工具，帮我添加 这个 Skills: <https://clawhub.ai/wwlyzzyorg/ppt-generator>
- 2 并且启用！

代码块

- 1 重启一下项目，帮我生成带token的登录链接。

通用项目二次开发方法论

本文从具体项目的二次开发实践中抽象提炼，形成一套可复用的通用方法论，适用于任何开源项目或已有项目的二次开发场景。

第一阶段：源码获取与环境搭建

1.1 获取源码

确定目标项目的代码仓库地址（GitHub / GitLab / Gitee 等），使用 Git 克隆到本地工作目录：

代码块

```
1 git clone <仓库地址>
2 cd <项目目录>
```

关键决策点：本地开发 vs 远程服务器开发

- 本地开发**：适合资源消耗小、无 GPU 依赖的项目，开发调试更便捷。
- 远程服务器开发**：适合模型推理、大数据量、高算力需求的项目。推荐使用 Cursor / VS Code Remote SSH 连接远程开发。

1.2 环境依赖安装

- 阅读项目的 `README.md`、`INSTALL.md` 或 `docs/` 文档，了解依赖栈（Python / Node / Docker 等）。
- 使用官方推荐方式安装依赖（`pip install -r requirements.txt` / `npm install` / `docker-compose up`）。
- 注意 Python 版本、Node 版本等运行时版本要求，优先使用 **虚拟环境**（`venv` / `conda` / `nvm`）隔离。

第二阶段：项目启动与 MVP 验证

2.1 启动项目

按照文档说明启动项目，常见方式：

代码块

```
1 # Python 项目
2 python main.py / uvicorn app:app --reload
```

```
3
4 # Node 项目
5 npm run dev / npm start
6
7 # Docker 项目
8 docker-compose up -d
```

2.2 配置 Rule / 规则文件

大多数可配置项目都有 rule、config、prompt 或 .env 文件。在启动前务必：

- 复制 `.env.example` → `.env`，填写 API Key、数据库地址等必填项。
- 理解核心 rule/config 文件的作用，按需修改。
- 确认模型名称、端口、路径等配置是否与本地环境匹配。

2.3 MVP 核心功能验证（冒烟测试）

项目跑起来后，先不做任何修改，验证原始功能是否正常：

- **对话能力测试**：发几条基础消息，确认核心流程跑通。
- **干话能力测试**：尝试边界输入，观察异常处理。
- 记录基准行为，作为二次开发对比的基线。

第三阶段：需求分析（最重要的阶段）

3.1 明确"为什么要二次开发"

在动手改代码之前，回答以下三个问题：

1. **原项目缺什么？**（功能缺口）
2. **我的用户场景是什么？**（使用对象、核心流程）
3. **改动边界在哪里？**（哪些改，哪些不动）

3.2 需求分类

需求类型	描述	优先级
功能裁剪	去掉原项目不需要的功能模块	高
功能增强	在原有逻辑上扩展新能力	高
UI/UX 定制	改界面风格、交互方式	中
模型替换	接入自定义模型或 API	视情况
数据层改造	替换数据库、存储方案	视情况

3.3 形成开发 Roadmap

将需求拆解成可执行任务，按优先级排序，形成简易 Roadmap。推荐用飞书多维表格或看板管理。

第四阶段：项目结构梳理

4.1 读懂项目骨架

在改代码之前，先用 AI（Cursor / Claude / GPT）帮助你快速理解项目结构：

代码块

```
1  # Cursor 中可以直接问：
2  "这个项目的入口文件是哪个？请帮我梳理核心目录结构和数据流。"
```

4.2 绘制核心数据流

理解以下关键路径：

- 请求入口 → 路由 → 业务逻辑 → 数据存取 → 响应输出
- 找到与你改动目标最相关的文件和函数

4.3 标注"禁止触碰区"

识别项目中高风险、不可轻易修改的核心模块（如：鉴权逻辑、数据库 Schema、核心算法），做好标注，避免意外破坏稳定性。

第五阶段：版本管理与升级策略

5.1 建立独立开发分支

代码块

```
1 git checkout -b feature/my-customization
```

永远不要直接在 `main` / `master` 分支上改动，保留一条干净的"原版"基线。

5.2 跟踪上游版本升级

当原项目发布新版本时，需要决策是否跟进升级：

- **重新下载源码**：适合改动量较小、与原项目差异不大的情况。重新 clone 后，对比差异重新应用自己的改动。
- **Git Rebase / Merge**：适合改动量较大、需要精细合并的场景。

5.3 更新开发规范

版本升级后，同步更新：

- 依赖文件（`requirements.txt` / `package.json`）
- 配置文件（`.env`、`config.yaml`）
- 开发文档（记录本次升级的关键变更）

第六阶段：二次开发执行（Vibe Coding 实战）

6.1 项目启动（上下文注入）

在 Cursor 或其他 AI 编辑器中，开发前先注入项目上下文：

- 用 `@codebase` 或 `@file` 引用关键文件，让 AI 了解项目背景。
- 写一段简短的项目说明作为 System Prompt，放在 `.cursorrules` 或对话开头。
- 示例："这是一个基于 XXX 框架的对话机器人项目，我需要在 YYY 模块增加 ZZZ 功能，入口文件是 `main.py`。"

6.2 接入自定义 Model

通用接入模式（OpenAI 兼容接口）：

代码块

```
1 BASE_URL=https://your-model-api.com/v1
2 API_KEY=your-api-key
```

- 优先找项目中已有的模型配置入口（通常在 `.env` 或 `config` 文件）。
- 若不支持，找到模型调用层，封装适配器（Adapter Pattern）。

6.3 Vibe Coding 核心原则

- **小步快跑**：每次改动范围控制在一个功能点，改完立即测试。
- **描述意图而非代码**：告诉 AI "我想实现什么效果"，而不是"帮我写这个函数"。
- **保持可回滚**：每个稳定节点 `git commit`，随时可回退。
- **AI 生成 + 人工审核**：AI 负责生成代码框架，人工负责逻辑校验和边界处理。
- **遇到卡点先问 AI**：把报错信息、相关代码一起丢给 AI，快速定位问题。

6.4 常见二次开发场景速查

场景	推荐做法
新增 API 接口	找到路由文件，按已有接口格式仿写，注意鉴权中间件
修改前端 UI	找到对应组件文件，小改用 CSS override，大改直接替换组件
修改 Prompt / System Prompt	找到 prompt 模板文件或配置项，独立提取为可配置变量
接入第三方服务	新建 service/ 目录下的适配模块，不侵入核心逻辑
数据库 Schema 变更	使用 Migration 工具（Alembic / Prisma），不手动改表结构

通用检查清单

开始前

- ☐ 原项目 README 通读完毕
- ☐ 本地/远程环境已搭建，项目可正常启动
- ☐ MVP 核心功能验证通过
- ☐ 需求已明确，Roadmap 已制定
- ☐ 已创建独立开发分支

开发中

- ☐ 每个功能点改完即测试
- ☐ 关键节点 git commit（附清晰 commit message）
- ☐ 未引入新的硬编码配置（统一放 .env）
- ☐ 新增功能有基本的错误处理

上线前

- ☐ 完整功能回归测试
- ☐ 敏感信息（API Key 等）未提交到仓库
- ☐ 开发文档已更新
- ☐ 部署流程已验证（本地 → 测试环境 → 生产环境）

核心心法总结

🧠 不懂先跑，跑通再改；改前读懂，改后测试。

🔧 AI 是你的副驾驶，你是系统的主人。

📦 小步提交，随时可退；功能隔离，互不干扰。